Manufacturing records before model predictions

Manufacturing internship and follow-up engineering project

I completed a one-month manufacturing technology internship at Mintech, supporting OLED deposition-equipment manufacturing. My role was to assist with factory tasks and support OLED deposition-equipment manufacturing under my supervisor's direction.

My contribution to the follow-up

I supplied the internship account and set the goal of connecting the experience with engineering and AI in a separate report. I used AI tools to develop the supporting documents and software, supplied a reference design, and directed revisions so the report would explain my role alongside the technical results.

The resulting project asks what records are needed to compare two preparation methods. Its main output is a Python CSV validator, supported by a carrier and witness-coupon checklist, a paired comparison plan and a workbook.

When the tool withholds a comparison

The proposed schema connects a carrier and sample to their preparation conditions and measurement points. Before calculating a difference in thickness range, the validator checks required information, run consistency, matching conditions and the measurement-point set.

One software test changes a measurement-point identifier to P4. Although the two methods still have equal numbers of measurements, their point sets differ. The validator returns POINT_SET and withholds the pair. The different locations make the spatial ranges unsuitable for a matched comparison.

Another test repeats a run's preparation-error count across three point rows. The validator counts that value once per run. In the constructed input, A's two errors remain two, instead of becoming six through row-wise addition.

The package includes the record schema, checklist, paired comparison plan, executable validator and saved test fixtures. All 55 configured software tests produced the expected issue codes and accepted-pair counts.

Scope

The follow-up design, implementation and automated checks used AI tools after the internship. The demonstrated results are software tests and a separate synthetic-data analysis; field A/B observations are not included.

What the AI exercise adds

Compare the cost of different mistakes

A separate pipeline uses 4,000 generated 4M records to explore classification after a dataset has been structured. The evaluation set contains 60 positive and 940 negative cases, with a decision threshold of 0.5.

Model	Positives found / 60	False alarms / 940
LogisticRegression	50	119
LightGBM	26	29

LogisticRegression finds more positives while sending more negative cases for inspection. LightGBM produces fewer false alarms and misses more positives. Choosing a model requires the cost of a missed defect and an unnecessary inspection, alongside the evaluation scores.

The labels are generated from rules that share variables with the inputs. The exercise tests a reproducible classification workflow; it does not measure defect prediction on company production data.

How the two parts connect

Record validation determines whether inputs meet declared comparison rules. Model evaluation asks what can be inferred from a dataset and which mistakes it makes. The current checklist and classifier remain separate implementations. Connecting them would require traceable run-level records, independently observed labels and features available at a defined prediction time.

Evidence to inspect

The accompanying English engineering report explains the record structure, three validation cases, model errors and proposed field-trial conditions. The package also contains source code, constructed test inputs, saved outputs and execution metadata. The field-record template is blank; the numerical examples are constructed software inputs.

The validator, its test inputs and saved results allow the record-checking rules to be inspected and reproduced.